Gutbrod Ádám

Specifikáció

Kalandjáték

A játék ismertetése

A program célja egy egyszerű RPG játék megvalósítása console-ban, szöveges megjelenítéssel.

A játékos miután elindította a játékot elkészíti karakterét: nevet és egy class-t választ. A class fogja meghatározni karaktere alapvető statisztikát, minthogy mennyi élete, védelme stb.-je van.

Ezután az a célja, hogy a kezdő mezőről eljusson a cél mezőbe, miközben különböző szörnyekkel harcol és tárgyakat vesz fel, amelyek segíteni fognak a játék folyamán. A navigációhoz segítségére lesz egy karakteresen megjelenített térkép, amin nyomon követheti karakter pozícióját, valamint látja a szörnyeket, tárgyakat, célmezőt, illetve falakat (azon mezőket értjük fal alatt amikre nem lehet rálépni, illetve sem szörny, sem tárgy nincsen rajta).

Amikor a játékos nincsen harcban (ld. köv. bekezdés), lehetősége van 4 irányba mozogni. Ezt 4 irányban teheti meg: fel, le, jobbra, balra. Továbbá ekkor használhatja tárgyait (pl.: valamilyen védelmi értéket növelő bájital). A mozgáson felül megtekinthető a statisztikáit. Erre szüksége lehet a játékosnak, hogy nyomon tudja követni, ahogy az egyes szintlépesekkel (ld. köv. bekezdés) fejlődik, erősödik.

Amennyiben a játékos egy olyan mezőre kerül, amin egy szörny tartózkodik, akkor meg kell küzdenie vele. A küzdelem egy körökre osztott rendszerben zajlik: az első kört a játékos kezdi egy támadás akcióval (pl.: light attack), ezt követően a szörny támad. Így zajlanak a körök mindaddig, ameddig a játékos nem győz, vagy meg nem hal. Ha a játékos meghalna (azaz életpontjai elérnék a 0-t), akkor véget ér a játék. Amennyiben győzedelmeskedik jutalmul XP-t (tapasztalati pontot) kap. Ha összegyűjt egy bizonyos mennyiséget szintet lép, ekkor lehetősége van egy valamilyen alap statisztikai értéket növelni (pl.: megnöveli a maximálás életpontjai).

A játék irányítása / kezelése

A játékot a megnyíló console-ban különböző karakterekkel lehet irányítani. A képernyőn minden esetben fel lesz sorolva a különböző akciók végrehajtásához társított karakter. Például amikor a játékos a karakterét készíti fel lesznek sorolva a különböző class-ok. Ezek mellett egyértelműen jelezve lesz, hogy milyen karakterrel lehet kiválasztani. A parancs megadása úgy történik, hogy begépeljük a karaktert, majd egy ENTER megnyomásával kiadjuk a parancsot. A megfelelő karakterek mindig indikálva, vagy expliciten (Press ’S’ to start) vagy impliciten (Start [S]).

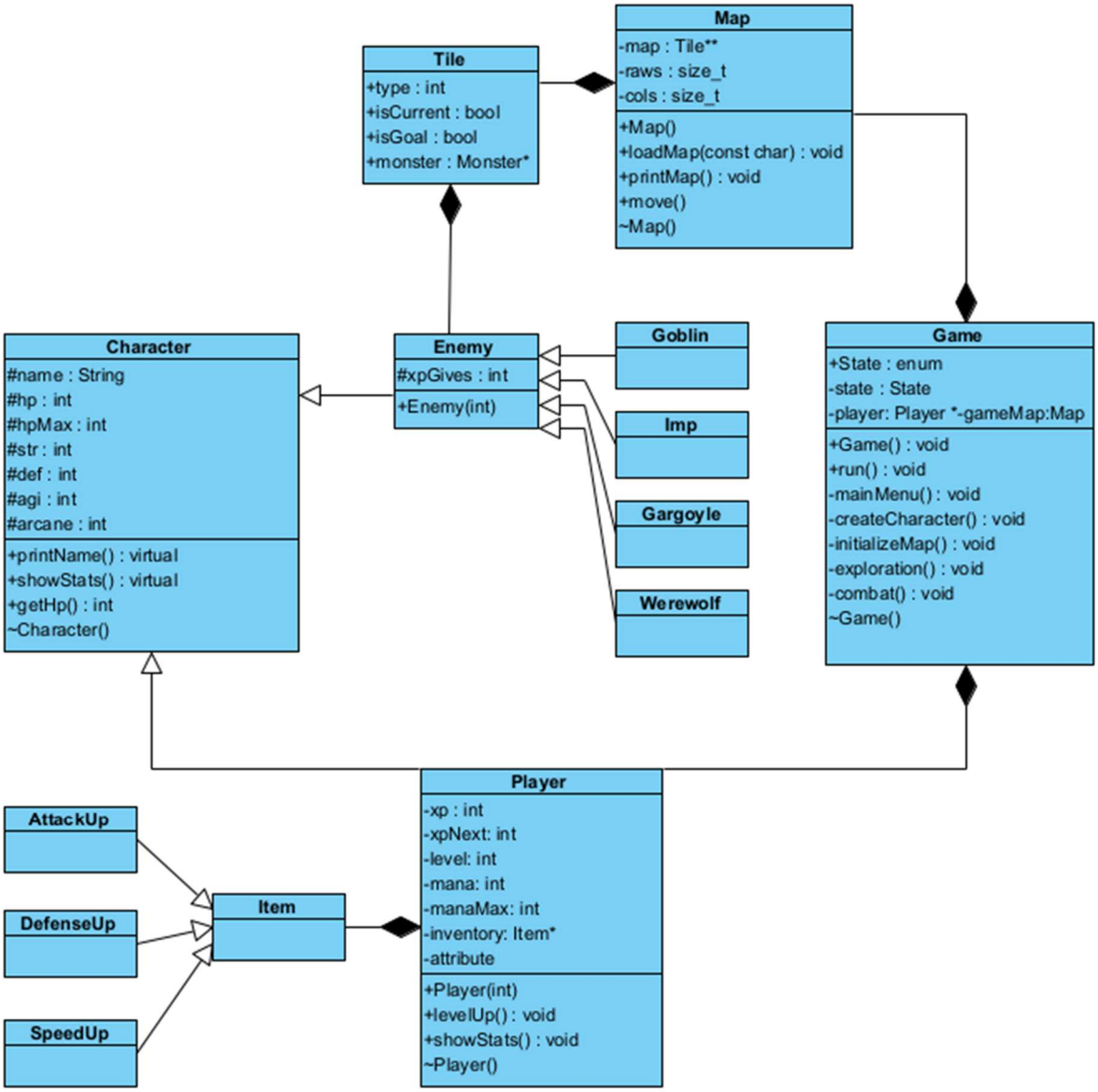
Egy esetben kell több karaktert írni, amikor a karakterünket nevezzük el. Ilyenkor a megadni kívánt név begépelése után nyomunk ENTER-t.

Fájlkezelés

A pálya betöltéséhez szükséges egy szöveges (txt) fáj, amely tartalmazza a megfelelő paramétereket, amelyekből a program megalkotja a játékteret. Ez a pálya előre van definiálva.

A fájl első sora tartalmazza a pálya méretét (sor és oszlop szám), a további sorok pedig SPACE-szel elválasztott számokat tartalmaznak, amit a program beolvasás után értelmez és előállítja belőle a pályát.

UML osztálydiagram



Fontosabb függvények

A pálya betöltése – `initializeMap(const char*) : void`

Meghívja a `Map::loadMap(const char*)` függvényt ami egy szóközzel tagolt szöveges fájlból beolvassa a pályát oly módon, hogy az első sor tartalmazza a pálya méretére vonatkozó paramétereket (sor/oszlop szám), illetve a további sorokban minden egész szám alapján létrehoz egy mezőt (`Tile`) és ezeknek a mátrixát fogja eltárolni. Ezeknek mezőknek beállítjuk a típusát, mint fal vagy mező, vagy szörnyet tartalmazó mező. Szörny esetén létrehozunk egy objektumot és egy erre mutató pointert tárolunk el (különben `nullptr`). Ezenkívül flag-ekkel van jelezni, hogy éppen az aktuális mezőn állunk-e, illetve melyik a cél mezőbe, amibe el kell jutni.

A játék futtatása – `run() : void`

A run() fogja lebonyolítani a program állapota között való váltást.

1. A program elindítását követően a menübe kerülünk (`menu state`). Innen kilépni lehet vagy elindítani a játékot.
2. A játék elindítását követően a karakter kreálás történik(`charactercreation state`). Ha itt valamilyen memória foglalási hiba történni, a program egy hibával tér vissza, különben a pálya betöltése következik (`mapinitization`).
3. A pálya betöltése közben több hiba előfordulhat, pl. nem létezik a betölteni kívánt .txt fájl, illetve itt is memória foglalási hiba is történhet. Mindkettő esetben hibával tér vissza a program és terminál.
4. Miután minden inicializálási folyamat sikeresen lezajlott elkezdődik a játék (`exploration state`). Innen csak a harcba (`combat state`) lehet átlépni, illetve amennyiben elérjük a cél mezőt nyerünk.
5. A harc kimenetelétől függően visszatérhetünk felfedezési állapotba vagy amennyiben meghalunk gameovert (`gameover state`)kapunk majd felszabadítjuk a lefoglalt memórát.

Harc – `combat() : void`

A harc 1 az egy ellen fog zajlani körökre osztottan, ahol a játékos kezdheti elsőként a körét. Különböző támadásokból választhat, úgymint *Light Attack*, *Heavy Attack*, *Fireball* stb. Ezek különböző statisztik alapján fejtik ki a hatásukat. A *Heavy Attack* bár sokat sebez, lassú, ezért kisebb eséllyel fog betalálni. A szörnyeknek egy-két alaptámadásuk lesz. Ezek közül véletlen alapon fognak választani, nem fogják figyelembe venni a játékos attribútumait, hogy adott esetben optimálisabban támadjanak. Egy körben egy akció megengedett.

Felfedezés, barangolás – `exploration() : void`

A játék ezen szakaszában lehet mozogni a pályán. Ezt karakteresen kiírt pálya segíti majd. Itt jelölve lesz a játékos aktuális helyzete ('P'), a falak ('#'), üres terek (' '), szörnyek ('M'), tárgyak ('*') [a karakterek nem feltétlen fogják a végleges állapotot tükrözni]. Mozogni lehet fel, le, jobbra, balra. Minden mozgás után frissül, majd kiíródik a térkép.

A mozgáson felül a játékos használhat bájitalokat (*Potion*), melyek ideiglenesen megnövelik az életét, sebzését stb.

Abban az esetben, ha a karakter egy szörny típusú mezőre lép elkezdődik a harc. Ha a cél mezőre ér, véget ér a játék.